COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	August 7, 2026	**DATE SUBMITTED:**	August 8, 2026
NAME:	DE VERA, SOphia D.	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

HANDS-ON ACTIVITY 5.1 QUEUES

1. PROCEDURE OUTPUT

ILO A

SCREENSHOT

ILOA_main.cpp

```
    };

    std::queue<std::string> a;

    // Insert students into the queue
    for (int i = 0; i < 6; i++) {
        a.push(Nstudents[i]);
    }

    std::cout << "The queue is: ";
    display(a);

    // Checking if the quueue is empty
    std::cout << "Is the queue empty?      " << a.empty() << std::endl;

    // Asking how many is in the queue
    std::cout << "How many is in the queue?      " << a.size() <<  std::endl;

    // The front letter
    std::cout << "What is the front letter?      " << a.front() <<  std::endl;
```

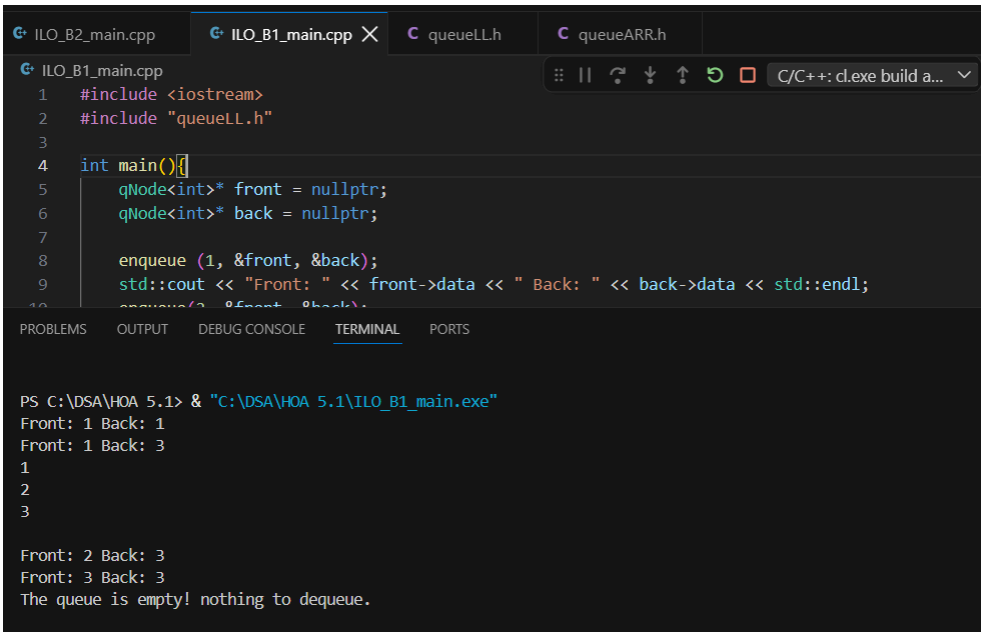
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\DSA\HOA 5.1>
PS C:\DSA\HOA 5.1>
PS C:\DSA\HOA 5.1> & "C:\DSA\HOA 5.1\ILOA_main.exe"
The queue is:  S O P H I A
Is the queue empty?     0
How many is in the queue?     6
What is the front letter?     S
What is the last letter?     A
Popping the front letters . . .
Current queue in line is:  H I A
The new queue a is:  H I A   C P E
```

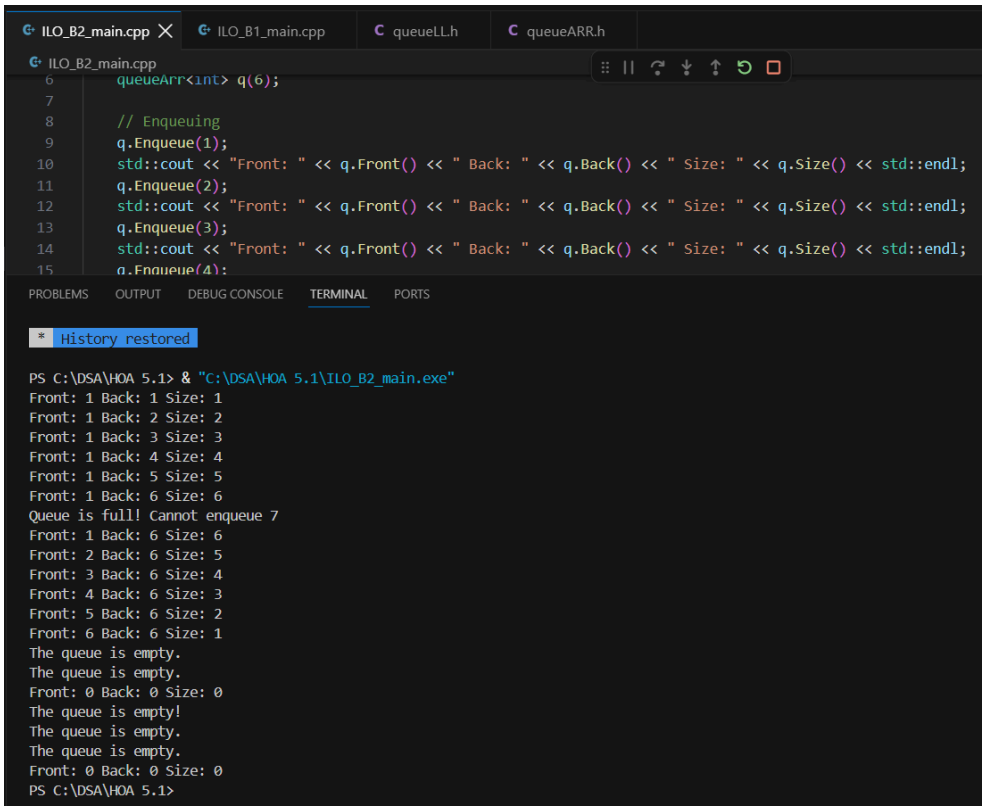
OBSERVATION ANALYSIS	- **While doing the activity, I noticed that the sequence of creating a queue is similar to linked listing and stacking. Queues follows the principle or FIFO**

	(first-in, first-out) because while creating the popping "function", I can't remove the first elements in front without popping them first (I had some errors while popping the middle letters). - The operation demonstrates the process by making the elements go in order based on their line. Each time pop() is performed, the queue becomes shorter and the next element moves to the front.

ILO B1

SCREENSHOT	
OBSERVATION ANALYSIS	- On this one, I successfully demonstrated the FIFO principle like on the ILO A. But with an additional task of managing the nodes and elements. Since the *.h* file used is a linked list, each element is stored in a node that is connected to the next node through a pointer. - I used the operations ***enqueue()*** and ***dequeue()*** adding and removing elements from the front and back

ILO B2

SCREENSHOT	
OBSERVATION ANALYSIS	- The ILO B2's instructions were used to make a queue using a dynamically allocated circular array, which puts a limit of capacity. We used 6 elements to fill the capacity. When the ***enqueue()*** is called, the value is added to the back of the queue, while ***q_size*** is increased to keep track of the number of elements. - We also used ***dequeue()*** to remove elements from the queue. The element: ***q_front*** acts as the "pointer to point the element next in line." - We also used the modulo sign to move the front and back positions circularly (because we were doing a circular array queue). - By the end of the code, we used ***clear()*** to reset the functions, front, back, and size values.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SCREENSHOT

SuppleAct_main.cpp | SPA_QLL.h

```
int main() {
#include "SPA_QLL.h"

int main() {

    qNode* front = nullptr;
    qNode* back = nullptr;

    Printing p1(1, "Sophia", 5);
    Printing p2(2, "Anna", 10);
    Printing p3(3, "Elsa", 3);
    Printing p4(4, "Cinderella", 8);
    Printing p5(5, "Spider-Man", 12);

    std::cout << "          PENDING PRINTING DOCUMENTS        " << std::endl;
    std::cout << std::endl;

    enqueue(p1, &front, &back);
    enqueue(p2, &front, &back);
    enqueue(p3, &front, &back);
    enqueue(p4, &front, &back);
    enqueue(p5, &front, &back);

    std::cout << "          PENDING PRINTING JOBS          " << std::endl;

    display(front);
    std::cout << "          PROCESSING JOBS         " << std::endl;

    while (front != nullptr) {
        std::cout << "Now printing . . . " << std::endl;
        front->data.display();
        std::cout << "You may now get your document(s)." << std::endl;
        std::cout << std::endl;

        dequeue(&front, &back);
    }
```

```
PS C:\DSA\HOA 5.1> & "C:\DSA\HOA 5.1\SuppleAct_main.exe"
          PENDING PRINTING DOCUMENTS

          PENDING PRINTING JOBS
Print Order: 1
From: Sophia
No. of Pages: 5


Print Order: 2
From: Anna
No. of Pages: 10


Print Order: 3
From: Elsa
No. of Pages: 3


Print Order: 4
From: Cinderella
No. of Pages: 8


Print Order: 5
From: Spider-Man
No. of Pages: 12
```

```
          PROCESSING JOBS
Now printing . . .
Print Order: 1
From: Sophia
No. of Pages: 5
You may now get your document(s).

Now printing . . .
Print Order: 2
From: Anna
No. of Pages: 10
You may now get your document(s).

Now printing . . .
Print Order: 3
From: Elsa
No. of Pages: 3
You may now get your document(s).

Now printing . . .
Print Order: 4
From: Cinderella
No. of Pages: 8
You may now get your document(s).

Now printing . . .
Print Order: 5
From: Spider-Man
No. of Pages: 12
You may now get your document(s).

          No more doucments to print. On standby.
```

OBSERVATION ANALYSIS

- To simulate a printer, I used the First-In, First-Out principle. Using a linked-list program, every printing request is stored in a separate node and connected to the next pending request. Any new document in the queue goes at the back of the pending list and the dequeue automatically calls the next document once the front one is finished.
- **DEFENDING**: As said in the instructions, I quote: *"first come, first served. Whoever submits the job first will be the first to get the job done."*
- So I used linked lists because, first, linked lists can have a flexible capacity while arrays have a fixed capacity. Thinking about it, when printing, you don't get to put a limit on the printing size. Second,linked lists are suitable for this situation

	because the printer only needs to add new jobs at the back and remove completed jobs from the front. With enqueue() and dequeue(), the printing jobs can be processed in the exact order they were submitted.

REFLECTION:

- **I kind of had a hard time doing this because i have a lot of things to consider while doing the code/program, especially on the .h file.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

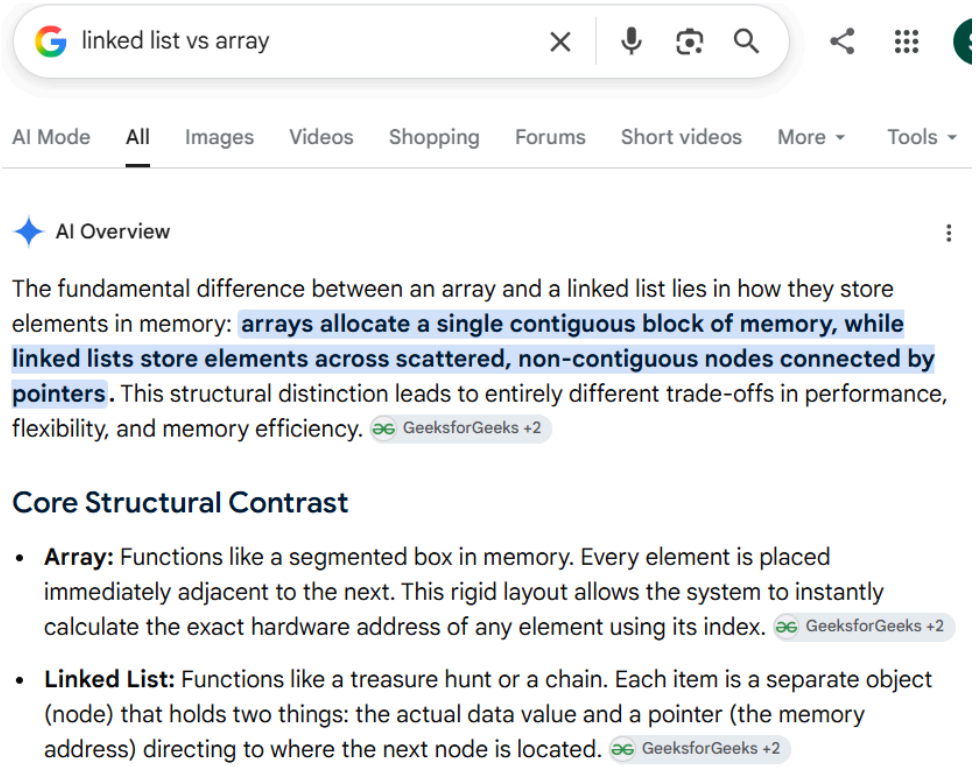
I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation and explainer.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.



Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.